

■ Docker + ■■ Kubernetes (AKS)

Complete Azure Cloud Deployment Guide

React • Spring Boot • Azure SQL • ACR • AKS

Docker Multi-Stage Builds • Docker Compose (Local) • AKS Cluster • Helm Charts

Kustomize Overlays • HPA Autoscaling • NetworkPolicy • Workload Identity

Azure DevOps YAML Pipelines • Trivy Security Scanning • Key Vault Integration

■ Backend Dockerfile	■ Frontend Dockerfile	■■ Docker Compose	■■ AKS Cluster
■ Azure ACR	■■ Helm Chart	■ Kustomize	■ Workload Identity
■ HPA Autoscaler	■■ NetworkPolicy	■ Key Vault	■ DevOps Pipelines

Table of Contents

PART 1 — DOCKER FUNDAMENTALS

- Ch 1: Why Docker? Containers vs VMs
- Ch 2: Backend Dockerfile — Multi-Stage Spring Boot Build
- Ch 3: Frontend Dockerfile — Multi-Stage React + Nginx
- Ch 4: Docker Compose — Full Local Stack

PART 2 — AZURE CONTAINER REGISTRY (ACR)

- Ch 5: Create ACR and Push Docker Images
- Ch 6: ACR Security — RBAC and Managed Identity

PART 3 — AZURE KUBERNETES SERVICE (AKS)

- Ch 7: AKS Architecture and Cluster Setup
- Ch 8: Kubernetes Manifests Deep Dive (Deployments, Services, Ingress)
- Ch 9: Autoscaling — HPA, PDB, TopologySpread
- Ch 10: Security — NetworkPolicy, Workload Identity, Key Vault

PART 4 — KUSTOMIZE & HELM

- Ch 11: Kustomize — Base + Dev/Prod Overlays
- Ch 12: Helm Chart — Templated Kubernetes Manifests

PART 5 — INFRASTRUCTURE AS CODE

- Ch 13: Bicep Template — AKS + ACR + SQL + Key Vault
- Ch 14: CLI Script — Complete AKS Deployment

PART 6 — CI/CD PIPELINES

- Ch 15: Pipeline Setup — Variable Groups & Service Connections
- Ch 16: Frontend Pipeline — React → Docker → ACR → AKS
- Ch 17: Backend Pipeline — Spring Boot → Docker → ACR → AKS
- Ch 18: Security Scanning with Trivy

PART 7 — OPERATIONS

- Ch 19: Monitoring — AKS + App Insights + Log Analytics
- Ch 20: Troubleshooting, Rollback & Cleanup

PART 1

Docker Fundamentals

Containerise the application — Dockerfiles, multi-stage builds, Docker Compose

Ch 1: Why Docker? Containers vs VMs

Ch 2: Backend Dockerfile — Spring Boot

Ch 3: Frontend Dockerfile — React + Nginx

Ch 4: Docker Compose — Full Local Stack

CHAPTER 1

Why Docker? Containers vs VMs

Understanding the foundation before we containerise our application.

1.1 The Problem Docker Solves

Without Docker: 'It works on my machine!' is the most infamous phrase in software. Your laptop runs Java 17 but the server has Java 11. Your teammate's Mac runs fine but the Linux CI server fails. Docker packages the application AND its entire runtime environment into a single portable unit called a container.

Aspect	Virtual Machine	Docker Container
Size	Several GB (full OS)	MBs (shares host OS kernel)
Startup time	Minutes	Seconds or milliseconds
Isolation	Full hardware virtualisation	Process-level (namespaces + cgroups)
Portability	Partial (hypervisor dependent)	Full (runs anywhere Docker runs)
Density	~10s per host	~1000s per host
Dev/Prod parity	Hard to achieve	Built-in — same image everywhere

1.2 Key Docker Concepts

- **Dockerfile** — Recipe for building an image. A text file with step-by-step instructions.
- **Image** — The built snapshot of your app + all dependencies. Immutable, versioned.
- **Container** — A running instance of an image. Lightweight, isolated, disposable.
- **Registry** — Where images are stored. Docker Hub is public; Azure ACR is private.
- **Layer cache** — Each Dockerfile instruction creates a cached layer. Unchanged layers are reused → fast rebuilds.
- **Multi-stage build** — Multiple FROM stages. Final image only contains what the last stage copies. Keeps images small and secure.

1.3 Our Application Container Strategy

Component	Base Image (Build)	Base Image (Runtime)	Final Size
Spring Boot Backend	maven:3.9.6-eclipse-temurin-17	eclipse-temurin:17-jre-jammy	~220 MB
React Frontend	node:18-alpine	nginx:1.25-alpine	~25 MB
Local SQL Server	—	mcr.microsoft.com/mssql/server:2022-latest	~1.5 GB (dev only)

■ Multi-stage builds are critical for production. A single-stage Spring Boot build image would be ~600MB. Our multi-stage approach produces a ~220MB final image by including only the JRE runtime, not the full JDK + Maven.

1.4 Docker in Our Workflow

- **Local dev** → docker compose up → runs React + Spring Boot + SQL Server all together
- **CI pipeline** → docker build + docker push → builds image and pushes to Azure ACR
- **AKS deployment** → kubectl apply → AKS pulls image from ACR and runs containers as pods
- **Profile selection** → SPRING_PROFILES_ACTIVE env var switches H2/SQL Server/Azure SQL

-DskipTests in package	Tests run in a dedicated pipeline step, not in every Docker build
JRE (not JDK) in runtime	JRE is ~200MB vs JDK ~400MB. No compiler needed at runtime.
Non-root user	Security best practice — limits blast radius if container is compromised
-XX:+UseContainerSupport	Without this, JVM reads total HOST RAM not container RAM limit → OOMKilled
-XX:MaxRAMPercentage=75.0	Prevents heap from consuming all RAM (OS + non-heap need ~25%)

2.3 Build and Run Manually

Build & Run Backend Container

```
# Build the image (from repo root)
docker build -t emp-backend:local ./backend

# Run with H2 (local profile – no DB needed)
docker run -p 8080:8080 \
-e SPRING_PROFILES_ACTIVE=local \
emp-backend:local

# Test
curl http://localhost:8080/api/health

# Run with SQL Server (docker profile)
docker run -p 8080:8080 \
-e SPRING_PROFILES_ACTIVE=docker \
-e AZURE_SQL_URL='jdbc:sqlserver://host.docker.internal:1433;database=employeeedb;...' \
-e AZURE_SQL_USERNAME=sa \
-e AZURE_SQL_PASSWORD='LocalDev@Pass123!' \
emp-backend:local

# Inspect the image layers
docker history emp-backend:local
docker image inspect emp-backend:local | python3 -m json.tool
```



```
location ~* \.(js|css|png|svg|ico|woff2?)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
# SPA routing – THIS IS CRITICAL
# Without this, refreshing /employees/5 returns 404
# With this, Nginx serves index.html → React Router takes over
location / {
    try_files $uri $uri/ /index.html;
}
# Health check for Kubernetes probes
location /health {
    return 200 "healthy\n";
    add_header Content-Type text/plain;
}
}
```

3.3 Build and Test

```
Build & Run Frontend Container
# Build for dev (API at localhost:8080)
docker build \
    --build-arg REACT_APP_API_URL=http://localhost:8080/api \
    -t emp-frontend:local ./frontend
# Run
docker run -p 3000:80 emp-frontend:local
# Open: http://localhost:3000
# Build for Azure (API at real Azure URL)
docker build \
    --build-arg REACT_APP_API_URL=https://api.emp.yourdomain.com/api \
    --build-arg GENERATE_SOURCEMAP=false \
    -t emp-frontend:prod ./frontend
# Check final image size
docker image ls emp-frontend
# Should be ~25-30MB – much smaller than the 500MB+ Node build image!
```

CHAPTER 4

Docker Compose — Full Local Stack

One command to run the entire application locally: React + Spring Boot + SQL Server.

4.1 docker-compose.yml Overview

Docker Compose orchestrates multiple containers as a single application. Our compose file defines 3 services that start in the right order, connected on a shared network.

Service	Image	Port	Profile
db	mcr.microsoft.com/mssql/server:2022	1433	—
backend	emp-backend:local (built)	8080	docker
frontend	emp-frontend:local (built)	80, 3000	—

4.2 Key docker-compose.yml Sections Explained

docker-compose.yml key sections

```
services:
db:
  image: mcr.microsoft.com/mssql/server:2022-latest
  environment:
    ACCEPT_EULA: 'Y'
    SA_PASSWORD: 'LocalDev@Pass123!'
    MSSQL_PID: 'Express' # Free edition
  volumes:
    - sqlserver_data:/var/opt/mssql # Data persists between restarts
  healthcheck:
    test: sqlcmd -S localhost -U sa -P 'LocalDev@Pass123!' -Q 'SELECT 1'
    interval: 15s
    retries: 10
    start_period: 30s
  backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  environment:
    SPRING_PROFILES_ACTIVE: docker # Uses application-docker.yml
    AZURE_SQL_URL: 'jdbc:sqlserver://db:1433;database=employeedb;...'
    # 'db' is the service name – Docker DNS resolves it!
  depends_on:
    db:
      condition: service_healthy # Wait for SQL Server to be ready
  frontend:
  build:
  args:
    REACT_APP_API_URL: http://localhost:8080/api
  depends_on:
```

```
backend:
condition: service_healthy # Wait for backend to be ready
```

4.3 Running the Local Stack

```
Docker Compose Commands
# Start everything (first time: builds images too)
docker compose up --build
# Start in detached mode (background)
docker compose up --build -d
# View live logs from all services
docker compose logs -f
# View logs for one service only
docker compose logs -f backend
# Check status of all services
docker compose ps
# Test the running stack
curl http://localhost:8080/api/health # Backend
curl http://localhost:8080/api/employees # API
curl http://localhost:3000 # Frontend
# Run a command inside a container
docker compose exec backend java -version
docker compose exec db sqlcmd -S localhost -U sa -P 'LocalDev@Pass123!' -Q 'SELECT name FROM sys.databases'
# Rebuild only one service after code change
docker compose up --build backend
# Stop services (keeps volumes)
docker compose down
# Stop AND delete all data (fresh start)
docker compose down -v
# Scale a service to 3 instances
docker compose up --scale backend=3
```

4.4 Spring Profiles in Docker Context

Profile	DB Used	How to Activate	When to Use
local	H2 in-memory	mvnw spring-boot:run -Dspring-boot.run.profiles=local	Pure Java dev, no Docker
docker	SQL Server container	docker compose up	Local full-stack with Docker
azure	Azure SQL Database	SPRING_PROFILES_ACTIVE=azure (in AKS/pipeline)	Production AKS

PART 2

Azure Container Registry (ACR)

Your private Docker registry in Azure — push, pull, and secure images

Ch 5: Create ACR and Push Images

Ch 6: ACR Security — RBAC & Managed Identity

CHAPTER 5

Create ACR and Push Docker Images

Azure Container Registry — your private Docker Hub in Azure.

5.1 Why Azure Container Registry?

- **Private** — images are only accessible to authorised users and services (unlike Docker Hub public repos)
- **Geo-replicated** — Premium tier replicates images to multiple regions for fast pulls globally
- **Integrated with AKS** — attach ACR to AKS cluster with one command; AKS pulls images without credentials
- **Vulnerability scanning** — built-in Microsoft Defender for Containers scans all pushed images
- **Tasks** — ACR can build Docker images directly in the cloud (no local Docker needed)

5.2 Create ACR — Portal

1	Search Container registries Search 'Container registries' in the portal → '+ Create'.
2	Basics Resource group: rg-emp-aks-dev. Registry name: empacr{uniquesuffix} (5-50 lowercase alphanumeric, globally unique). Location: East US. Pricing plan: Basic (dev) / Premium (prod for geo-replication).
3	Networking Public access (for simplicity). Premium tier supports private endpoints.
4	Review + Create Click 'Review + create' → 'Create'. Takes ~30 seconds.
5	Note the login server Copy the 'Login server' value: empacr123456.azurecr.io — this is your image registry prefix.

5.3 Create ACR — CLI

```
Create ACR
# Create ACR
ACR_NAME='empacr'$(openssl rand -hex 3)
az acr create \
  --name $ACR_NAME \
  --resource-group rg-emp-aks-dev \
  --location eastus \
  --sku Basic \
  --admin-enabled false # Use managed identity, not admin creds
# Get the login server URL
ACR_SERVER=$(az acr show --name $ACR_NAME --query loginServer -o tsv)
echo "ACR Login Server: $ACR_SERVER"
# e.g.: empacr1a2b3c.azurecr.io
```

5.4 Build, Tag and Push Images

```
Push Images to ACR
# Login to ACR (uses your Azure CLI session – no password needed!)
az acr login --name $ACR_NAME

# Build and tag backend
docker build -t $ACR_SERVER/emp-backend:v1.0.0 ./backend
docker build -t $ACR_SERVER/emp-backend:latest ./backend

# Push backend image to ACR
docker push $ACR_SERVER/emp-backend:v1.0.0
docker push $ACR_SERVER/emp-backend:latest

# Build and push frontend
docker build \
--build-arg REACT_APP_API_URL=https://api.emp.yourdomain.com/api \
-t $ACR_SERVER/emp-frontend:v1.0.0 \
./frontend
docker push $ACR_SERVER/emp-frontend:v1.0.0

# List images in ACR
az acr repository list --name $ACR_NAME --output table

# List tags for an image
az acr repository show-tags \
--name $ACR_NAME \
--repository emp-backend \
--orderby time_desc \
--output table

# Alternative: Build directly in ACR (no local Docker needed!)
az acr build \
--registry $ACR_NAME \
--image emp-backend:v1.0.0 \
./backend
```


CHAPTER 6

ACR Security — RBAC and Managed Identity

Grant AKS permission to pull images from ACR without storing passwords.

6.1 The AcrPull Role

AKS needs to pull images from ACR. Instead of storing ACR credentials in Kubernetes secrets (insecure), we grant the AKS cluster's kubelet identity the AcrPull role on the ACR resource. The kubelet (the agent on each AKS node) uses this identity to pull images automatically.

```
Grant AKS access to ACR
# Attach ACR to AKS (simplest method – recommended!)
# This grants AcrPull role to AKS kubelet identity
az aks update \
--resource-group rg-emp-aks-dev \
--name aks-emp-dev \
--attach-acr $ACR_NAME
# Verify: list role assignments on ACR
az role assignment list \
--scope $(az acr show --name $ACR_NAME --query id -o tsv) \
--output table
# Alternative: Grant manually (if AKS already created)
ACR_ID=$(az acr show --name $ACR_NAME --query id -o tsv)
KUBELET_ID=$(az aks show -g rg-emp-aks-dev -n aks-emp-dev \
--query 'identityProfile.kubeletidentity.objectId' -o tsv)
az role assignment create \
--assignee $KUBELET_ID \
--role 'AcrPull' \
--scope $ACR_ID
```

■ Once --attach-acr is set, your Kubernetes Deployment manifests can reference ACR images directly (e.g., myacr.azurecr.io/emp-backend:v1) and AKS pulls them automatically — no imagePullSecrets needed!

6.2 Image Tag Strategy

Strategy	Example	When to Use
Build ID tag	emp-backend:1542	Unique per pipeline run. Use in production for traceability.
Semantic version	emp-backend:v1.2.3	For versioned releases. Use alongside build ID.
Branch-latest	emp-backend:main-latest	Floating tag for dev/staging. Quick deployment.
Latest	emp-backend:latest	Never use in production — not reproducible!

PART 3

Azure Kubernetes Service (AKS)

Deploy, scale, and secure your containers with Kubernetes

Ch 7: AKS Architecture & Cluster Setup

Ch 8: Kubernetes Manifests Deep Dive

Ch 9: Autoscaling — HPA, PDB, TopologySpread

Ch 10: Security — NetworkPolicy & Workload Identity

CHAPTER 7

AKS Architecture and Cluster Setup

Azure Kubernetes Service — managed Kubernetes so you manage apps, not control planes.

7.1 AKS Architecture

AKS is a managed Kubernetes service. Microsoft manages the control plane (API server, etcd, scheduler, controller manager). You only manage the node pools (the VMs running your containers).

Component	Description	Managed By
Control Plane	API server, etcd, scheduler	Microsoft (free!)
Node Pool (System)	VM nodes running kube-system pods	You pay for VMs
Node Pool (User)	VM nodes running your app pods	You pay for VMs
Azure Load Balancer	External traffic entry point	Azure (auto-created)
Azure CNI	Pod networking — each pod gets VNet IP	Azure
OIDC Issuer	Federated identity for Workload Identity	Microsoft

7.2 Create AKS Cluster — Portal

1	Search Kubernetes services Search 'Kubernetes services' → '+ Create' → 'Create a Kubernetes cluster'.
2	Basics Resource group: rg-emp-aks-dev. Cluster name: aks-emp-dev. Region: East US. Kubernetes version: 1.29. Authentication: System-assigned managed identity.
3	Node pools System node pool: Standard_D2s_v3, 1 node (dev). Enable autoscaler: min 1, max 3.
4	Networking Network configuration: Azure CNI. Load balancer: Standard. Enable HTTP application routing: OFF (we'll install our own NGINX ingress).
5	Integrations Container registry: select your ACR (this grants AcrPull). Azure Monitor: ON → select your Log Analytics workspace.
6	Advanced Enable OIDC issuer: YES ■ (required for Workload Identity). Enable Workload Identity: YES ■.
7	Review + Create Create. Takes 5-10 minutes. Go get a coffee ■

8

Get credentials

After creation: `az aks get-credentials --resource-group rg-emp-aks-dev --name aks-emp-dev`

7.3 Create AKS — CLI (Recommended)

```
Create AKS Cluster
az aks create \
--resource-group rg-emp-aks-dev \
--name aks-emp-dev \
--location eastus \
--node-count 1 \
--node-vm-size Standard_D2s_v3 \
--enable-cluster-autoscaler \
--min-count 1 --max-count 3 \
--enable-oidc-issuer \ # For Workload Identity
--enable-workload-identity \ # For Key Vault access
--network-plugin azure \ # Azure CNI
--generate-ssh-keys \
--attach-acr $ACR_NAME \ # Grant AcrPull automatically
--workspace-resource-id $LAW_ID \ # Send logs to Log Analytics
--enable-addons monitoring
# Get kubeconfig credentials
az aks get-credentials \
--resource-group rg-emp-aks-dev \
--name aks-emp-dev \
--overwrite-existing
# Verify connection
kubectl get nodes
kubectl get namespaces
kubectl cluster-info
```

7.4 Install NGINX Ingress Controller

```
Install NGINX Ingress + cert-manager
# Add helm repo
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
# Install ingress controller
helm upgrade --install ingress-nginx ingress-nginx/ingress-nginx \
--namespace ingress-nginx \
--create-namespace \
--set controller.replicaCount=2 \
--wait --timeout 5m
# Get the external IP (wait 1-2 minutes for Azure LoadBalancer)
kubectl get svc -n ingress-nginx
INGRESS_IP=$(kubectl get svc ingress-nginx-controller \
-n ingress-nginx -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "Ingress IP: $INGRESS_IP"
# Install cert-manager for automatic TLS certificates
kubectl apply -f
https://github.com/cert-manager/cert-manager/releases/download/v1.14.4/cert-manager.yaml
kubectl wait --for=condition=available deployment \
-l app.kubernetes.io/instance=cert-manager \
```

```
-n cert-manager --timeout=120s
```

CHAPTER 8

Kubernetes Manifests Deep Dive

Every YAML field explained: Deployments, Services, Ingress, ConfigMaps, Secrets.

8.1 Namespace

```
namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: employee-app
  labels:
    app.kubernetes.io/name: employee-management
# All our resources go in this namespace – isolated from system pods
```

8.2 Deployment — Key Fields Explained

```
Deployment — key fields
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: employee-app
spec:
  replicas: 2 # 2 pods – if 1 dies, 1 still serves traffic
  selector:
    matchLabels:
      app: backend # Which pods this deployment manages
  strategy:
    type: RollingUpdate
  rollingUpdate:
    maxSurge: 1 # Bring up 1 new pod BEFORE killing old ones
    maxUnavailable: 0 # NEVER take a pod down unless replacement is ready
  template: # Pod template – what each pod looks like
    spec:
      containers:
        - name: backend
          image: myacr.azurecr.io/emp-backend:1542
          resources:
            requests: # Guaranteed – scheduler uses this for placement
              cpu: 250m # 0.25 vCPU
              memory: 512Mi
            limits: # Hard cap – container killed if exceeded
              cpu: 1000m # 1 vCPU max
              memory: 1Gi # 1GB max
```

8.3 The Three Health Probes

Probe	Purpose	Failure Action	Our Settings
-------	---------	----------------	--------------

startupProbe	Is the app done starting?	Keep waiting (don't kill yet)	30s delay, 12 retries = 2min max
livenessProbe	Is the app alive / not frozen?	Restart the container	Every 20s, fail after 3 misses
readinessProbe	Can the app handle requests?	Remove from Service (no traffic)	Every 10s, fail after 3 misses

Health Probes

All three probes call Spring Boot Actuator endpoints:

startupProbe: # /actuator/health (generic – app is started)

httpGet:

path: /actuator/health

port: 8080

initialDelaySeconds: 30 # Give Spring Boot time to start

periodSeconds: 10

failureThreshold: 12 # 12 * 10s = 2 minutes before giving up

readinessProbe: # /actuator/health/readiness (is DB connected?)

httpGet:

path: /actuator/health/readiness

port: 8080

initialDelaySeconds: 30

periodSeconds: 10

failureThreshold: 3

livenessProbe: # /actuator/health/liveness (is app loop working?)

httpGet:

path: /actuator/health/liveness

port: 8080

initialDelaySeconds: 90 # Only check after startup probe passes

periodSeconds: 20

failureThreshold: 3

8.4 Service — ClusterIP vs LoadBalancer

Service Type	Accessible From	Use Case
ClusterIP	Inside cluster only	Default — backend service (reached via Ingress)
NodePort	Node IP + port	Dev/testing only
LoadBalancer	Internet (Azure creates ALB)	Simple public exposure (bypasses Ingress)
Ingress	Internet via NGINX controller	Production — single IP, routing by host/path

8.5 Ingress — Routing Rules

ingress.yaml

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

annotations:

cert-manager.io/cluster-issuer: letsencrypt-prod # Auto TLS!

nginx.ingress.kubernetes.io/ssl-redirect: 'true'


```
nginx.ingress.kubernetes.io/limit-rps: '100' # Rate limiting
spec:
tls:
- hosts: [emp.yourdomain.com, api.emp.yourdomain.com]
secretName: emp-tls-secret # cert-manager populates this
rules:
- host: emp.yourdomain.com # Frontend traffic
http:
paths:
- path: /
backend:
service:
name: frontend-service
port:
number: 80
- host: api.emp.yourdomain.com # Backend API traffic
http:
paths:
- path: /api
backend:
service:
name: backend-service
port:
number: 80
```

CHAPTER 9

Autoscaling — HPA, PDB, TopologySpread

Scale up automatically under load. Guarantee minimum availability during maintenance.

9.1 Horizontal Pod Autoscaler (HPA)

HPA watches CPU and memory metrics and automatically adjusts the number of pod replicas. When your app gets a traffic spike at 9 AM, HPA adds pods within 60 seconds. When traffic drops, it removes them to save cost.

```
hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: employee-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 2 # Never go below 2 (HA even at zero traffic)
  maxReplicas: 10 # Never create more than 10 pods
  metrics:
    - type: Resource
      resource:
        name: cpu
      target:
        type: Utilization
        averageUtilization: 70 # Scale up when avg CPU > 70%
    - type: Resource
      resource:
        name: memory
      target:
        type: Utilization
        averageUtilization: 80 # Scale up when avg Memory > 80%
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60 # Wait 60s after last scale-up
    policies:
      - type: Pods
        value: 2
        periodSeconds: 60 # Add max 2 pods per minute
    scaleDown:
      stabilizationWindowSeconds: 300 # Wait 5 min before scaling down
    policies:
      - type: Pods
        value: 1
        periodSeconds: 120 # Remove max 1 pod per 2 minutes
```

9.2 Pod Disruption Budget (PDB)

PDB tells Kubernetes: 'when doing voluntary disruptions (node upgrades, draining), always keep at least 1 pod running.' Without this, a node upgrade could terminate all pods simultaneously.

```

pdb.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: backend-pdb
  namespace: employee-app
spec:
  minAvailable: 1 # Keep at least 1 backend pod running always
  selector:
    matchLabels:
      app: backend
  # Azure uses this when: node pool upgrades, node drains,
  # cluster upgrades, spot node evictions

```

9.3 TopologySpreadConstraints — Spread Pods Across Nodes

Without spread constraints, Kubernetes might put all 3 replicas on the same node. If that node fails, all pods die. TopologySpread forces pods onto different nodes:

```

TopologySpreadConstraints
topologySpreadConstraints:
  - maxSkew: 1 # Max 1 more pod on any node vs others
  topologyKey: kubernetes.io/hostname # Spread across different nodes
  whenUnsatisfiable: DoNotSchedule # Don't schedule if spread impossible
  labelSelector:
    matchLabels:
      app: backend
  # Result: with 3 replicas on 3 nodes, each node gets 1 pod
  # If 1 node dies, only 1 pod dies, 2 remain serving traffic

```

9.4 Monitor Scaling

```

Monitor HPA
# Watch HPA in action
kubectl get hpa -n employee-app -w
# See current replicas and targets
kubectl describe hpa backend-hpa -n employee-app
# View pod distribution across nodes
kubectl get pods -n employee-app -o wide
# Generate load to trigger HPA (test)
kubectl run load-test --image=busybox --rm -it -- \
/bin/sh -c 'while true; do wget -q -O- http://backend-service/api/employees; done'
# Watch HPA respond to load
kubectl get hpa -n employee-app -w

```

CHAPTER 10

Security — NetworkPolicy, Workload Identity, Key Vault

Zero-trust networking, passwordless Azure access, and centralised secret management.

10.1 NetworkPolicy — Pod-to-Pod Firewall

By default, all pods in a cluster can talk to all other pods. NetworkPolicy creates firewall rules at the pod level. We restrict: frontend → backend (allowed), backend → internet (restricted), direct access to backend bypassing frontend (denied).

```
network-policy.yaml
# Allow only frontend pods to call backend
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: employee-app
spec:
  podSelector:
    matchLabels:
      app: backend # This policy applies TO backend pods
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend # Allow FROM frontend pods
      - podSelector:
            matchLabels:
              app.kubernetes.io/name: ingress-nginx # Allow FROM Ingress
      ports:
        - port: 8080
    policyTypes:
      - Ingress
# Result: ONLY frontend and ingress controller can reach backend port 8080
# Any other pod trying to call backend gets: Connection refused
```

10.2 Workload Identity — Passwordless Azure Access

Workload Identity lets Kubernetes pods authenticate to Azure services (like Key Vault) without storing any credentials. It uses OIDC token federation between AKS and Azure AD.

- **Step 1** — Create a User-Assigned Managed Identity in Azure.
- **Step 2** — Create a Kubernetes ServiceAccount with an annotation linking to the MI.
- **Step 3** — Create a Federated Credential: 'When AKS pod uses backend-sa ServiceAccount, trust it as this Managed Identity'.
- **Step 4** — Grant the Managed Identity 'Key Vault Secrets User' role on Key Vault.
- **Step 5** — Backend pods labelled azure.workload.identity/use: 'true' automatically get Azure tokens!

```
Setup Workload Identity
# 1. Create Managed Identity
az identity create --name mi-emp-dev --resource-group rg-emp-aks-dev
MI_CLIENT_ID=$(az identity show -g rg-emp-aks-dev -n mi-emp-dev --query clientId -o tsv)

# 2. Get OIDC Issuer URL from AKS
OIDC_URL=$(az aks show -g rg-emp-aks-dev -n aks-emp-dev \
--query 'oidcIssuerProfile.issuerUrl' -o tsv)

# 3. Create Federated Credential
az identity federated-credential create \
--name aks-backend-fedcred \
--identity-name mi-emp-dev \
--resource-group rg-emp-aks-dev \
--issuer $OIDC_URL \
--subject 'system:serviceaccount:employee-app:backend-sa' \
--audiences api://AzureADTokenExchange

# 4. Grant Key Vault access to the MI
KV_ID=$(az keyvault show -n $KV_NAME -g rg-emp-aks-dev --query id -o tsv)
MI_PRINCIPAL=$(az identity show -g rg-emp-aks-dev -n mi-emp-dev --query principalId -o tsv)
az role assignment create --assignee $MI_PRINCIPAL \
--role 'Key Vault Secrets User' --scope $KV_ID

# 5. Kubernetes ServiceAccount (references the MI)
# serviceaccount.yaml:
# metadata:
# annotations:
# azure.workload.identity/client-id: $MI_CLIENT_ID
```

10.3 K8s Secrets — Injected by Pipeline from Key Vault

Kubernetes Secrets store base64-encoded data. We create them in the pipeline by fetching values from Key Vault — secrets never touch source code:

```
Pipeline: KV → K8s Secret
# In the pipeline (azure-pipelines-backend-k8s.yml):
# Fetch all values from Azure Key Vault
KV_NAME=$(az keyvault list -g $RG --query '[0].name' -o tsv)
SQL_URL=$(az keyvault secret show --vault-name $KV_NAME \
--name AzureSqlUrl --query value -o tsv)

# Create/Update Kubernetes Secret from Key Vault values
kubectl create secret generic emp-secrets \
--namespace employee-app \
--from-literal=AZURE_SQL_URL="$SQL_URL" \
--from-literal=AZURE_SQL_USERNAME="$SQL_USER" \
--from-literal=AZURE_SQL_PASSWORD="$SQL_PASS" \
--from-literal=APPLICATIONINSIGHTS_CONNECTION_STRING="$APPI" \
--dry-run=client -o yaml | kubectl apply -f -
# --dry-run=client + kubectl apply = idempotent (safe to run multiple times)
```


PART 4

Kustomize & Helm

Manage manifests across environments without YAML duplication

Ch 11: Kustomize — Base + Dev/Prod Overlays

Ch 12: Helm Chart — Templated Manifests

CHAPTER 11

Kustomize — Base + Dev/Prod Overlays

Manage Kubernetes manifests for multiple environments without duplication.

11.1 What is Kustomize?

Kustomize lets you have a base set of Kubernetes manifests and overlay changes for each environment — without templating languages or duplicating YAML. It's built into kubectl (kubectl apply -k).

Kustomize	Helm
No new language — pure YAML + patches	Go templates — powerful but complex
Built into kubectl	Requires Helm install
Great for simple env differences	Great for distributable packages
Patches using JSON patch or strategic merge	Full templating with if/else/loops
Our use: dev=1 replica, prod=3 replicas	Our use: parameterise entire chart

11.2 Directory Structure

```

Kustomize Structure
k8s/
  ■■■ base/ # Shared manifests — same for all envs
  ■■■ kustomization.yaml # Lists all resources
  ■■■ namespace.yaml
  ■■■ backend-deployment.yaml
  ■■■ frontend-deployment.yaml
  ■■■ ingress.yaml
  ■■■ hpa.yaml
  ■■■ pdb.yaml
  ■■■ network-policy.yaml
  ■■■ configmap.yaml
  ■■■ secret.yaml
  ■■■ serviceaccount.yaml
  ■■■ overlays/
  ■■■ dev/
  ■■■ kustomization.yaml # Patches: 1 replica, dev image tag
  ■■■ prod/
  ■■■ kustomization.yaml # Patches: 3 replicas, prod image tag

```

11.3 Base kustomization.yaml

```

base/kustomization.yaml
# k8s/base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: employee-app

```



```
resources:
- namespace.yaml
- serviceaccount.yaml
- configmap.yaml
- secret.yaml
- backend-deployment.yaml
- frontend-deployment.yaml
- ingress.yaml
- hpa.yaml
- pdb.yaml
- network-policy.yaml
```

11.4 Dev Overlay

```
overlays/dev/kustomization.yaml
# k8s/overlays/dev/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
bases:
- ../../base
# Override replicas to 1 for dev (save cost)
patches:
- patch: |-
- op: replace
path: /spec/replicas
value: 1
target:
kind: Deployment
name: backend
- patch: |-
- op: replace
path: /spec/replicas
value: 1
target:
kind: Deployment
name: frontend
# Point to dev image tags
images:
- name: REGISTRY_PLACEHOLDER/emp-backend
newTag: dev-latest
- name: REGISTRY_PLACEHOLDER/emp-frontend
newTag: dev-latest
```

11.5 Using Kustomize

```
Kustomize Commands
# Preview what will be deployed (dry run)
kubectl kustomize k8s/overlays/dev
# Deploy dev environment
kubectl apply -k k8s/overlays/dev
# Deploy prod environment
kubectl apply -k k8s/overlays/prod
# Delete all resources for dev
kubectl delete -k k8s/overlays/dev
```

```
# Verify deployed resources
kubectl get all -n employee-app
```

CHAPTER 12

Helm Chart — Templated Kubernetes Manifests

Package your entire application as a reusable, version-controlled Helm chart.

12.1 Helm Chart Structure

```
Helm Chart Structure
helm/employee-app/
■■■ Chart.yaml # Metadata: name, version, description
■■■ values.yaml # Default values for all environments
■■■ values-prod.yaml # Production overrides
■■■ templates/
■■■ backend.yaml # Deployment + Service + HPA (Go-templated)
■■■ frontend.yaml # Deployment + Service + HPA
■■■ ingress.yaml # Ingress (conditional on values.ingress.enabled)
■■■ secret.yaml # K8s Secret (values injected at deploy time)
■■■ serviceaccount.yaml # ServiceAccount (conditional)
```

12.2 Key Helm Template Syntax

```
Helm Template Syntax
# Referencing values.yaml fields:
replicas: {{ .Values.backend.replicaCount }}
# Constructing image name from multiple values:
image: "{{ .Values.global.registry }}/{{ .Values.backend.image.repository }}:{{ .Values.global.imageTag }}"
# Conditional block (only render if enabled):
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
...
{{- end }}
# Embed a YAML block from values (resources object):
resources:
{{- toYaml .Values.backend.resources | nindent 12 }}
# Loop over items:
{{- range .Values.backend.envVars }}
- name: {{ .name }}
value: {{ .value | quote }}
{{- end }}
```

12.3 Install and Upgrade with Helm

```
Helm Commands
# Install (first deployment)
helm install emp ./helm/employee-app \
--namespace employee-app \
--create-namespace \
--set global.registry=myacr.azurecr.io \
```

```
--set global.imageTag=1542 \  
--set secrets.azureSqlUrl='jdbc:sqlserver://...' \  
--set secrets.azureSqlPassword='YourPass!'  
# Upgrade (subsequent deployments)  
helm upgrade emp ./helm/employee-app \  
--namespace employee-app \  
--set global.imageTag=1600  
# Deploy with prod values file  
helm upgrade --install emp ./helm/employee-app \  
--namespace employee-app \  
-f helm/employee-app/values-prod.yaml \  
--set global.imageTag=$BUILD_ID \  
--set secrets.azureSqlPassword=$SQL_PASS  
# List installed releases  
helm list -n employee-app  
# Check release history  
helm history emp -n employee-app  
# Rollback to previous version  
helm rollback emp 1 -n employee-app  
# Uninstall everything  
helm uninstall emp -n employee-app  
# Dry-run (show what would be deployed)  
helm upgrade --install emp ./helm/employee-app --dry-run --debug
```


PART 5

Infrastructure as Code

Bicep template and CLI script for the full AKS infrastructure

Ch 13: Bicep Template — Complete AKS Stack

Ch 14: CLI Script — Automated Deployment

CHAPTER 13

Bicep Template — AKS + ACR + SQL + Key Vault

One Bicep file deploys the entire AKS infrastructure declaratively.

13.1 Resources Deployed by the Bicep Template

Resource	Bicep Type	Purpose
Log Analytics Workspace	Microsoft.OperationallInsights/workspaces	Central log storage
Application Insights	Microsoft.Insights/components	App performance monitoring
Azure Container Registry	Microsoft.ContainerRegistry/registries	Store Docker images
User-Assigned Managed Identity	Microsoft.ManagedIdentity/userAssignedIdentities	Passwordless AKS→KV auth
AKS Cluster	Microsoft.ContainerService/managedClusters	Kubernetes runtime
Azure SQL Server	Microsoft.Sql/servers	Database host
Azure SQL Database	Microsoft.Sql/servers/databases	Employee data
Key Vault	Microsoft.KeyVault/vaults	Secrets management
Key Vault Secrets (x4)	Microsoft.KeyVault/vaults/secrets	SQL URL, user, pass, APPI
RBAC: AcrPull	Microsoft.Authorization/roleAssignments	AKS can pull from ACR
RBAC: KV Secrets User	Microsoft.Authorization/roleAssignments	MI can read KV secrets

13.2 AKS Cluster Bicep Block

```
AKS Bicep resource
resource aks 'Microsoft.ContainerService/managedClusters@2024-01-01' = {
name: aksName
location: location
identity: { type: 'SystemAssigned' }
properties: {
kubernetesVersion: '1.29'
// workload Identity requires both of these
oidcIssuerProfile: { enabled: true }
securityProfile: { workloadIdentity: { enabled: true } }
agentPoolProfiles: [{
name: 'system'
count: environment == 'prod' ? 3 : 1
vmSize: 'Standard_D2s_v3'
```

```
enableAutoScaling: true
minCount: 1
maxCount: environment == 'prod' ? 10 : 3
}]
addonProfiles: {
omsagent: {
enabled: true
config: { logAnalyticsWorkspaceResourceID: law.id }
}
}
}
```

13.3 Deploy Bicep

```
Deploy Bicep Template
# Create resource group
az group create --name rg-emp-aks-dev --location eastus
# Validate (dry-run)
az deployment group validate \
--resource-group rg-emp-aks-dev \
--template-file infrastructure/bicep/aks-main.bicep \
--parameters environment=dev sqlAdminPassword='EmpP@ss2024!'
# What-if preview
az deployment group what-if \
--resource-group rg-emp-aks-dev \
--template-file infrastructure/bicep/aks-main.bicep \
--parameters environment=dev sqlAdminPassword='EmpP@ss2024!'
# Deploy!
az deployment group create \
--resource-group rg-emp-aks-dev \
--template-file infrastructure/bicep/aks-main.bicep \
--parameters environment=dev sqlAdminPassword='EmpP@ss2024!' \
--name deploy-aks-dev-001
# Get outputs
az deployment group show \
--resource-group rg-emp-aks-dev \
--name deploy-aks-dev-001 \
--query properties.outputs
```


CHAPTER 14

CLI Script — Complete AKS Deployment

The deploy-aks.sh script deploys all 12 steps automatically.

14.1 Script Overview

Step	What It Does	CLI Command
1	Create Resource Group	az group create
2	Log Analytics + App Insights	az monitor log-analytics workspace create + az monitor app-insights component create
3	Azure Container Registry	az acr create
4	AKS Cluster (5-10 min)	az aks create --enable-oidc-issuer --enable-workload-identity --attach-acr
5	User-Assigned Managed Identity	az identity create
6	Azure SQL Server + Database	az sql server create + az sql db create
7	Key Vault + 4 Secrets	az keyvault create + az keyvault secret set (x4)
8	Federated Identity Credential	az identity federated-credential create
9	NGINX Ingress Controller	helm upgrade --install ingress-nginx
10	cert-manager (TLS)	kubectl apply -f cert-manager.yaml
11	K8s Namespace + Secret	kubectl create namespace + kubectl create secret
12	Summary + aks-outputs.env	echo outputs to file

14.2 Run the Script

```

Run Deploy Script
# Deploy dev environment
bash infrastructure/cli-scripts/deploy-aks.sh dev

# Deploy prod environment
bash infrastructure/cli-scripts/deploy-aks.sh prod

# After script completes, view the saved outputs
cat aks-outputs.env

# Example aks-outputs.env content:
# ACR_SERVER=empacr1a2b3c.azurecr.io
# ACR_NAME=empacr1a2b3c
# AKS_NAME=aks-emp-dev
# RESOURCE_GROUP=rg-emp-aks-dev
# KV_NAME=kv-emp-dev-1a2b3c
# MI_CLIENT_ID=12345678-1234-...
# SQL_FQDN=emp-sql-dev-1a2b3c.database.windows.net

```

```
# Source the outputs into your shell
source aks-outputs.env
# Verify cluster is ready
kubectl get nodes
kubectl get svc -n ingress-nginx
```


PART 6

CI/CD Pipelines

Azure DevOps YAML pipelines: Docker build → ACR push → AKS deploy

Ch 15: Pipeline Setup

Ch 16: Frontend Pipeline Deep Dive

Ch 17: Backend Pipeline Deep Dive

Ch 18: Trivy Security Scanning

CHAPTER 15

Pipeline Setup — Variable Groups & Service Connections

Configure Azure DevOps before running the Docker + AKS pipelines.

15.1 Create Service Connection

1	Project Settings Azure DevOps → Project Settings → Service connections → + New service connection.
2	Azure Resource Manager Select 'Azure Resource Manager' → 'Service principal (automatic)'.
3	Configure Subscription: yours. Resource group: leave blank. Name: sc-azure-emp. Grant access to all pipelines: ■. Click Save.

15.2 Create Variable Group 'aks-global-vars'

1	Open Library Pipelines → Library → + Variable group. Name: aks-global-vars.
2	Add variables Click + Add variable for each: • ACR_NAME = empacr1a2b3c (your ACR short name) • AKS_NAME = aks-emp-dev • RESOURCE_GROUP = rg-emp-aks-dev • BACKEND_API_URL_DEV = https://api.emp-dev.yourdomain.com/api • BACKEND_API_URL_PROD = https://api.emp.yourdomain.com/api • K8S_NAMESPACE = employee-app
3	Pipeline permissions Pipeline permissions → + Add → your pipelines. Click Save.

15.3 Create Pipeline Environments with Approval Gates

1	Create environments Pipelines → Environments → + New environment for each: • frontend-dev-aks → Resource: None • frontend-prod-aks → Resource: None • backend-dev-aks → Resource: None • backend-prod-aks → Resource: None
2	Add approval to prod envs Click 'frontend-prod-aks' → Approvals and checks → '+' → Approvals → add your email → Timeout: 24h. Repeat for backend-prod-aks.

15.4 Create Both Pipelines

1

Frontend pipeline

New pipeline → Azure Repos Git → your repo → Existing YAML →
/pipelines/azure-pipelines-frontend-k8s.yml → Save and run.

2

Backend pipeline

New pipeline → Same steps → /pipelines/azure-pipelines-backend-k8s.yml → Save and run.

CHAPTER 16

Frontend Pipeline — React → Docker → ACR → AKS

4-stage pipeline: Test → Docker Build → Deploy Dev → Deploy Prod (with approval).

16.1 Pipeline Stages Summary

Stage	Jobs	Key Steps	Runs When
Test	TestJob	npm ci → npm test → publish coverage	Every push + PR
BuildImage	DockerBuildPush	docker build → docker push to ACR → Trivy scan	Not on PRs
DeployDev	DeployFrontendDev	Get AKS creds → kubectl apply -k overlays/dev → smoke test	Not on PRs
DeployProd	DeployFrontendProd	Rebuild with prod API URL → push prod tag → kubectl apply -k overlays/prod	main branch only + approval

16.2 Docker Build in Pipeline — Key Details

- **ACR Login** — `az acr login --name $ACR_NAME` uses the service connection's SP credentials automatically. No docker login needed!
- **Build Args for env-specific API URL** — `REACT_APP_API_URL` is baked into the JavaScript at build time. Dev and prod need different values → we rebuild in each stage.
- **Two image tags** — We push `$(Build.BuildId)` (permanent, immutable) AND `$(branchname)-latest` (floating). Production deployments always use the build ID tag.
- **Platform flag** — `--platform linux/amd64` ensures the image works on AKS Linux nodes even if the pipeline agent is ARM (e.g. M1/M2 Mac agents).

16.3 Deploy Step — Manifest Substitution

```
Pipeline: Substitute + Deploy
# In the pipeline deploy step:
ACR_SERVER=$(az acr show --name $(ACR_NAME) --query loginServer -o tsv)
# Replace PLACEHOLDER values in all YAML files
find k8s/ -name '*.yaml' -exec sed -i \
's|REGISTRY_PLACEHOLDER|'$ACR_SERVER'|g' {} \;
find k8s/ -name '*.yaml' -exec sed -i \
's|IMAGE_TAG_PLACEHOLDER|$(Build.BuildId)|g' {} \;
# Apply the dev overlay
kubectl apply -k k8s/overlays/dev
# Wait for rollout to complete
kubectl rollout status deployment/frontend \
-n $(K8S_NAMESPACE) --timeout=5m
# Verify
kubectl get pods -n $(K8S_NAMESPACE) -l app=frontend
```

CHAPTER 17

Backend Pipeline — Spring Boot → Docker → ACR → AKS

5-stage pipeline: Maven Build → Docker → Dev → Security Scan → Prod (with rollback).

17.1 Maven Build with H2 (No Azure needed in CI!)

The pipeline uses the 'local' Spring Boot profile for testing — H2 database runs in memory, zero external dependencies. This makes tests fast, reliable, and free.

```
Maven Test in Pipeline
# Pipeline YAML – Maven test with H2 profile
- task: Maven@4
  displayName: '■ Test with H2 (local profile)'
  inputs:
    mavenPomFile: backend/pom.xml
    goals: test
    options: '-Dspring.profiles.active=local'
    publishJUnitResults: true
    testResultsFiles: '**/surefire-reports/TEST-*.xml'
  # This runs ALL tests including integration tests
  # with H2 so no Azure SQL Server is needed in CI!
```

17.2 Multi-Platform Docker Build

```
Multi-platform Docker Build
# Use Docker Buildx for multi-platform builds
docker buildx create --use --name emp-builder 2>/dev/null || true
docker buildx build \
  --platform linux/amd64 \ # AKS nodes are AMD64
  --tag $ACR_SERVER/$(IMAGE_NAME):$(IMAGE_TAG) \
  --tag $ACR_SERVER/$(IMAGE_NAME):$(Build.SourceBranchName)-latest \
  --file $(DOCKERFILE_PATH) \
  --push \ # Push directly (no intermediate local image)
  $(BUILD_CONTEXT)
```

17.3 Syncing Secrets from Key Vault to K8s

```
Sync KV secrets to K8s
# Every deployment refreshes K8s secrets from Key Vault
# This ensures secret rotation is automatically picked up
KV_NAME=$(az keyvault list -g $(RESOURCE_GROUP) --query '[0].name' -o tsv)
# Fetch each secret from Key Vault
SQL_URL=$(az keyvault secret show --vault-name $KV_NAME \
  --name AzureSqlUrl --query value -o tsv)
SQL_USER=$(az keyvault secret show --vault-name $KV_NAME \
  --name AzureSqlUsername --query value -o tsv)
SQL_PASS=$(az keyvault secret show --vault-name $KV_NAME \
```



```
--name AzureSqlPassword --query value -o tsv)
APPI=$(az keyvault secret show --vault-name $KV_NAME \
--name AppInsightsConnectionString --query value -o tsv)
# Create/update Kubernetes secret idempotently
kubectl create secret generic emp-secrets \
--namespace $(K8S_NAMESPACE) \
--from-literal=AZURE_SQL_URL="$SQL_URL" \
--from-literal=AZURE_SQL_USERNAME="$SQL_USER" \
--from-literal=AZURE_SQL_PASSWORD="$SQL_PASS" \
--from-literal=APPLICATIONINSIGHTS_CONNECTION_STRING="$APPI" \
--dry-run=client -o yaml | kubectl apply -f -
```

17.4 Automatic Rollback on Failure

```
Automatic Rollback
# After production deployment:
# Port-forward to test internally (no public domain needed in pipeline)
kubectl port-forward svc/backend-service 8080:80 &
PF_PID=$!
sleep 15
STATUS=$(curl -s -o /dev/null -w '%{http_code}' http://localhost:8080/api/health)
kill $PF_PID 2>/dev/null || true
if [ "$STATUS" -eq 200 ]; then
echo '■ Deployment healthy!'
else
echo '■ Health check failed – rolling back!'
kubectl rollout undo deployment/backend -n $(K8S_NAMESPACE)
exit 1 # Fail the pipeline
fi
```

CHAPTER 18

Security Scanning with Trivy

Scan Docker images for CVEs before they reach production.

18.1 What is Trivy?

Trivy (by Aqua Security) is an open-source vulnerability scanner for container images, filesystems, and Kubernetes manifests. It checks the OS packages and application dependencies in your image against CVE databases.

18.2 Trivy in the Pipeline

```

Trivy in Pipeline
# Pipeline step – runs after docker push
- task: AzureCLI@2
  displayName: '■ Container Security Scan (Trivy)'
  continueOnError: true # Don't fail pipeline for warnings
  inputs:
    scriptType: bash
    inlineScript: |
      # Install Trivy
      curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
      | sh -s -- -b /usr/local/bin
      ACR_SERVER=$(az acr show --name $(ACR_NAME) --query loginServer -o tsv)
      # Scan for HIGH and CRITICAL vulnerabilities
      trivy image \
      --severity HIGH,CRITICAL \
      --format table \
      --exit-code 0 \ # 0 = report but don't fail
      $ACR_SERVER/$(IMAGE_NAME):$(IMAGE_TAG)
      # For strict mode (fail on CRITICAL):
      # --exit-code 1 --severity CRITICAL

```

18.3 Why Our Images Are Lean and Secure

- **Multi-stage build** — Final image has JRE only. No compiler, no build tools, no source code → fewer CVE targets.
- **Alpine/Jammy base** — Minimal OS. Alpine is especially small (~5MB) with fewer packages.
- **Non-root user** — appuser / nginx user. If the container is compromised, attacker has no root access.
- **No shell in prod** — Distroless images (alternative) have zero shell. Even harder to exploit.
- **GENERATE_SOURCEMAP=false** — React source maps removed from prod build. No code exposure in browser.

PART 7

Operations

Monitor, troubleshoot, rollback, and clean up

Ch 19: Monitoring — AKS + App Insights

Ch 20: Troubleshooting & Cleanup


```
// Failed requests in AKS-deployed backend (last 1h)
requests
| where timestamp > ago(1h)
| where success == false
| project timestamp, name, resultCode, duration, cloud_RoleInstance
| order by timestamp desc

// P95 response time per endpoint
requests
| where timestamp > ago(24h)
| summarize percentile(duration,95) by name
| order by percentile_duration_95 desc

// Container restart events (from Log Analytics)
KubeEvents
| where Namespace == 'employee-app'
| where Reason == 'BackOff' or Reason == 'OOMKilling'
| project TimeGenerated, Name, Message
| order by TimeGenerated desc
```

CHAPTER 20

Troubleshooting, Rollback & Cleanup

Diagnose common AKS issues and clean up resources to save costs.

20.1 Common Issues & Fixes

Problem	Symptom	Diagnosis	Fix
Image pull failure	Pod: ImagePullBackOff	kubectl describe pod → Events section	Check ACR name in manifest; verify AcrPull role; run az aks check-acr
OOMKilled	Pod restarts constantly	kubectl describe pod → Last State: OOMKilled	Increase memory limit in deployment resources.limits.memory
CrashLoopBackOff	Pod keeps restarting	kubectl logs --previous pod-name	Check app startup errors; verify env vars and secrets
Pending pod	Pod stuck in Pending	kubectl describe pod → Events: FailedScheduling	Check node resources (kubectl top nodes); add node pool capacity
KV secret ■	App can't read DB password	kubectl logs → 'Login failed for user'	Verify federated credential; check MI client ID in serviceaccount.yaml
Ingress 502	Browser: 502 Bad Gateway	kubectl logs -n ingress-nginx	Check backend pod is Running+Ready; verify service port in ingress.yaml

20.2 Rollback Procedures

```

Rollback Commands
# Rollback deployment to previous image version
kubectl rollout undo deployment/backend -n employee-app
kubectl rollout undo deployment/frontend -n employee-app
# Rollback to a specific revision
kubectl rollout history deployment/backend -n employee-app
kubectl rollout undo deployment/backend --to-revision=3 -n employee-app
# Rollback Helm release
helm history emp -n employee-app
helm rollback emp 2 -n employee-app # Rollback to revision 2
# Rollback pipeline: re-run the previous successful pipeline build
# In Azure DevOps: Pipelines → your pipeline → find previous run → Re-run

```

20.3 Complete Cleanup

Cleanup Commands

```
# Delete all K8s resources
kubectl delete -k k8s/overlays/dev
# or: kubectl delete namespace employee-app
# Uninstall Helm release
helm uninstall emp -n employee-app
# Delete NGINX ingress
helm uninstall ingress-nginx -n ingress-nginx
# Delete cert-manager
kubectl delete -f
https://github.com/cert-manager/cert-manager/releases/download/v1.14.4/cert-manager.yaml
# !! DELETE EVERYTHING – Resource Group !!
az group delete --name rg-emp-aks-dev --yes --no-wait
# This deletes: AKS, ACR, SQL, Key Vault, App Insights, MI – everything!
# Verify deletion (may take 5-10 minutes)
az group list --query '[?contains(name, `emp`)]' --output table
```

20.4 Cost Saving Tips for Dev

- **Stop AKS node pool at night** → `az aks nodepool scale --node-count 0` (free control plane!)
- **Use Spot VMs** → `--priority Spot --eviction-policy Delete` → 60-90% cheaper
- **B-series VMs** → Standard_B2s is burstable and much cheaper than D-series for dev
- **Delete dev resources on Friday** → `bash cleanup.sh` → recreate Monday morning
- **Free tier SQL** → Basic 5 DTU (\$5/month) is fine for dev/testing
- **ACR Basic SKU** → \$0.17/GB/month storage — tiny for dev images